Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 12
Calitate, performanță și pregătirea proiectului

Măsurăm aplicația cu Lighthouse, o îmbunătățim, construim versiunea de producție și pregătim README-ul, verificările finale și susținerea proiectului.

Tema	Temele 13 și 14 din planul disciplinei (examene.pe.ub.ro): calitatea aplicațiilor web și finalizarea proiectului
Durata	2 ore
Aveți nevoie de	aplicația proiectului, browserul Google Chrome și, pentru echipe, colegul de proiect
La final aveți	raportul Lighthouse înainte și după îmbunătățiri, folderul `dist`, README-ul proiectului și lista de verificare bifată; commit-ul final pe GitHub

1. Obiective

La finalul laboratorului veți putea:

- rula un audit Lighthouse și interpreta cele patru scoruri;
- aplica îmbunătățiri concrete de performanță, accesibilitate și SEO;
- construi versiunea de producție a interfeței și explica ce conține `dist`;
- pregăti README-ul, verificările finale și prezentarea proiectului.

2. Pe scurt: calitatea se măsoară

O aplicație „care merge” pe calculatorul autorului nu este neapărat o aplicație bună. **Lighthouse**, inclus în Chrome, verifică automat pagina și dă scoruri de la 0 la 100 pentru patru categorii [2]: **Performance** (cât de repede apare și răspunde pagina), **Accessibility** (dacă o pot folosi și persoanele cu dizabilități, conform WCAG [3]), **Best Practices** (erori în consolă, practici de securitate) și **SEO** (dacă motoarele de căutare înțeleg pagina). Culorile: verde 90–100, portocaliu 50–89, roșu 0–49.

Serverul Vite de la portul 5173 este gândit pentru dezvoltare: trimite codul neoptimizat, în multe fișiere. De aceea scorul **Performance** măsurat acolo este doar orientativ; versiunea de producție (secțiunea 6) este mult mai mică și mai rapidă [4]. Celelalte trei scoruri sunt relevante de pe acum [1].

3. Pornirea

1. Porniți **Docker Desktop** (mesajul **Engine running**). Comenzile se scriu în terminalul editorului, în folderul `daw-template-ti-info-2026-2027`; pentru probleme, vedeți metodologia [6].
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul proiectului (comun sau propriu), treceți pe el; apoi aduceți ultimele modificări și porniți containerele:

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul proiectului:
git checkout proiect-popescu-ionescu   # sau: git switch proiect-popescu-ionescu
git pull
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

4. Auditul Lighthouse

1. Deschideți Chrome într-o fereastră **Incognito** (**Ctrl + Shift + N**): extensiile instalate pot strica măsurătorile. Intrați pe http://localhost:5173.
2. **F12** → tab-ul **Lighthouse** (dacă nu se vede, căutați-l după săgeata »). Alegeți **Mode: Navigation**, **Device: Desktop** și bifați cele patru categorii. Apăsați **Analyze page load**.
3. Notați cele patru scoruri. Repetați pentru pagina **Produse** și pentru pagina unui produs. Salvați raportul (meniul ⋮ al raportului → **Save as HTML**).

Rezultat. Raportul afișează patru cercuri colorate. Sub fiecare categorie, la **Diagnostics** sau **Failed audits**, apar problemele găsite; click pe una deschide explicația și elementele HTML vinovate, iar **Learn more** duce la documentația ei.

Problemă tipică semnalată	Categoria	Ce faceți
Image elements do not have explicit width and height	Performance	puneți `width` și `height` pe imagini
Properly size images / Serve images in next-gen formats	Performance	imagini redimensionate, în format WebP
Background and foreground colors do not have a sufficient contrast ratio	Accessibility	culori cu contrast de cel puțin 4,5:1 pentru textul normal
Form elements do not have associated labels	Accessibility	fiecare câmp are o etichetă `<label>`
Document doesn't have a title element / meta description	SEO	titlu și descriere pentru fiecare pagină
Browser errors were logged to the console	Best Practices	corectați erorile din **Console**

5. Îmbunătățiri concrete

5.1 Imagini

Dacă aplicația afișează imagini (de exemplu, fotografiile produselor din proiectul vostru), micșorați-le la dimensiunea la care sunt afișate (o poză de 4000 px nu are ce căuta într-o cartelă de 300 px) și salvați-le ca **WebP**. În JSX, dați-le dimensiuni, text alternativ și încărcare amânată pentru cele care nu se văd de la început:

```
<img src={product.imageUrl} alt={product.name}
  width="300" height="200" loading="lazy" />
```

Sfat. `loading="lazy"` amână descărcarea imaginii până când utilizatorul ajunge, prin derulare, aproape de ea. Nu îl puneți pe imaginea principală din partea de sus a paginii: aceea trebuie să apară imediat. Înlocuiți `product.imageUrl` cu câmpul vostru de imagine.

5.2 Titluri de pagină și descriere

În React 19, o etichetă `<title>` scrisă oriunde în componentă este mutată automat în `<head>` [5]. Adăugați câte una în fiecare pagină din `pages/`, ca primul element din JSX-ul returnat, de exemplu în `ProductsPage.jsx`:

```
<title>Produse — Magazinul nostru</title>
```

În `frontend/app/index.html`, verificați că eticheta `<html>` are `lang="ro"` și adăugați în `<head>` o descriere:

```
<meta name="description"
  content="Magazin online demonstrativ: produse, categorii și recenzii.">
```

5.3 Contrast și etichete

- Verificați culorile textului pe fundal cu WebAIM Contrast Checker: pentru textul normal, cel puțin **4,5:1** (nivelul AA din WCAG [3]). Corectați în `App.css`.
- Fiecare `<input>`, `<select>` și `<textarea>` are o etichetă `<label>` (ca în `ReviewSection`). Butoanele au text vizibil; un buton doar cu pictogramă primește `aria-label`.
- Rulați din nou Lighthouse și comparați scorurile cu cele de la început.

6. Versiunea de producție (npm run build)

1. Construiți interfața în containerul React:
   ```
   docker exec -it daw-react npm run build
   ```
2. Deschideți folderul `frontend/app/dist` în editor: conține `index.html` și folderul `assets`, cu fișiere precum `index-…js` și `index-…css`.

Rezultat. Terminalul afișează `✓ built in …` și dimensiunea fiecărui fișier, inclusiv comprimată (*gzip*). Codul este **minificat** (fără spații și comentarii) și **grupat** în câteva fișiere; numele conțin un cod (*hash*) care se schimbă la fiecare modificare, ca browserul să nu folosească o versiune veche din cache [4].

`dist` este ceea ce se copiază pe un server web real. Valoarea `VITE_API_URL` este „scrisă” în cod în momentul construirii, deci pentru producție se construiește cu adresa API-ului de producție. Folderul `dist` **nu** se pune în Git: se poate genera oricând. Dacă `git status` îl arată, adăugați rândul `dist` în `frontend/app/.gitignore`.

7. README-ul proiectului

Pe branch-ul proiectului, înlocuiți conținutul fișierului `README.md` din rădăcină cu descrierea proiectului vostru (pe `main` rămâne README-ul template-ului). Puneți capturile de ecran în folderul `docs/capturi/`. Un schelet:

```
# Magazinul nostru — proiect PW/DAW 2026–2027

Aplicație web pentru ... (problema rezolvată, în 2–3 propoziții).

## Tehnologii
Symfony 8 (PHP 8.5), PostgreSQL 18, Doctrine ORM, React 19 + Vite, Docker Compose.

## Rulare
1. git clone https://github.com/cezarpapara/daw-template-ti-info-2026-2027.git
2. git checkout proiect-popescu-ionescu
3. docker compose -f backend/compose.yaml up -d --build
4. docker compose -f frontend/compose.yaml up -d --build
5. docker exec -it daw-php composer install
6. docker exec -it daw-php php bin/console doctrine:migrations:migrate
7. Datele de exemplu: backend/app/sql/date-exemplu.sql (rulat în DBeaver)
8. Interfața: http://localhost:5173 · API: http://localhost:8080/api/doc

## Capturi de ecran
![Pagina unui produs, cu recenzii](docs/capturi/produs.png)

## Contribuția fiecărui membru
- Ion Popescu: entitățile și API-ul pentru recenzii, validarea.
- Maria Ionescu: paginile React, formularele, auditul Lighthouse.
```

Atenție. Completați pașii de rulare exact cum sunt în metodologia template-ului [6] și verificați-i pe alt calculator (secțiunea 8). Dacă, la aducerea noutăților din `main`, apare un conflict în `README.md`, păstrați varianta voastră.

8. Lista de verificare înainte de susținere

- **Totul este pe GitHub**: `git status` nu mai arată modificări, iar ultimul `git push` a reușit.
- **Aplicația pornește de la zero pe alt calculator**: colegul (sau un coleg de grupă) clonează repository-ul, trece pe branch-ul proiectului și urmează **doar** pașii din README. Opriți înainte containerele `daw-*` care rulează deja pe acel calculator, ca să nu se ciocnească numele.
- **Migrările rulează** pe o bază de date goală, fără erori, iar `doctrine:schema:validate` este verde.
- **Datele de exemplu există**: scriptul `backend/app/sql/date-exemplu.sql` (cu `INSERT`-uri pentru produse, categorii și recenzii) este în Git și rulează în DBeaver (**Execute SQL Script**, **Alt + X**).
- **Nicio eroare** în Console sau Network la scenariul de demonstrație; codul de test (de exemplu `PasswordDemoController`) a fost șters.
- **README-ul** este complet, cu capturi și contribuții; scorurile Lighthouse (înainte și după) sunt notate în el.

9. Susținerea proiectului

Susținerea este fizică, în fața cadrelor didactice, și durează **10–15 minute** pentru o echipă. O structură care funcționează:

Partea	Durata	Ce arătați
Problema	1–2 min	ce rezolvă aplicația și pentru cine
Demonstrația	4–5 min	un scenariu pregătit dinainte, cu date de exemplu: listă, adăugare, modificare, ștergere, o eroare de validare
Arhitectura	2–3 min	drumul unei cereri (React → nginx → Symfony → PostgreSQL), entitățile și relațiile, rutele API în Swagger
Ce a făcut fiecare	2 min	fiecare membru prezintă partea lui; lista de commit-uri de pe GitHub ca dovadă
Întrebări	3–5 min	fiecare membru trebuie să poată explica orice parte a codului, nu doar pe a lui

După susținere, completați formularul **„AutoApreciere realizare proiect”** de pe platforma dlui Pruteanu (examene.pe.ub.ro), conform indicațiilor de acolo. Pregătiți-vă răspunsurile cinstit: ce funcționează, ce nu ați reușit și care a fost contribuția voastră personală.

10. Salvarea lucrului și oprirea

```
git add .
git commit -m "Laboratorul 12: imbunatatiri Lighthouse, README si date de exemplu"
git push origin proiect-popescu-ionescu
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

> **Push.** `git push origin proiect-popescu-ionescu` este forma completă; la proiectul individual, scrieți numele branch-ului vostru (`git push origin popescu.ion`). Forma scurtă, `git push`, merge doar dacă branch-ul local a fost legat anterior de cel de pe GitHub, prin `git push -u origin …` (laboratorul 11) sau, la colegul care l-a preluat, prin `git checkout`; `-u` leagă branch-ul local de cel de pe GitHub.

11. Exerciții

1. Aplicați cel puțin trei îmbunătățiri din secțiunea 5 și notați în README scorurile Lighthouse înainte și după.
2. Scrieți scriptul `date-exemplu.sql` și verificați lista din secțiunea 8 împreună cu un coleg din altă echipă, pe calculatorul lui.
3. ★ (opțional) Rulați auditul și cu **Device: Mobile** și corectați o problemă care apare doar acolo.

4. ★ (opțional) Înregistrați un video de 2–3 minute cu demonstrația, ca rezervă în caz că ceva nu merge la susținere.

12. Ce arătați la final

- raportul Lighthouse inițial și cel după îmbunătățiri, cu explicarea a două probleme corectate;
- folderul `dist` generat și README-ul proiectului afișat pe GitHub, pe branch-ul proiectului;
- lista de verificare din secțiunea 8, bifată, și commit-ul final pe GitHub.

Resurse pentru exersare

- **W3Schools – Accessibility:** https://www.w3schools.com/accessibility/
- **Curs HTML și CSS în limba română (playlist YouTube):** https://www.youtube.com/playlist?list=PL7aWL1nkGk0zssBjfDBtRMxtAGL3KNmAJ
- **Chrome – Lighthouse (documentație):** https://developer.chrome.com/docs/lighthouse/overview
- **WebAIM – Contrast Checker:** https://webaim.org/resources/contrastchecker/
- **Vite – Building for Production:** https://vite.dev/guide/build.html

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] Chrome for Developers, „Introduction to Lighthouse”. developer.chrome.com

[3] W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*, 2023. w3.org/TR/WCAG22

[4] Vite, „Building for Production”, documentație oficială. vite.dev

[5] React, „<title>”, documentație oficială. react.dev

[6] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*. Universitatea „Vasile Alecsandri” din Bacău, 2026.